

Blood Glucose Forecasting Benchmark - Structure Overview

This document provides an overview of the complete benchmark structure created for reproducible blood glucose prediction model comparison.

Created Components

1. Main Framework Structure

- **benchmark/**: Main benchmark package with modular design
- **benchmark/init.py**: Package initialization and main interface
- **benchmark/cli.py**: Command-line interface for running experiments

2. Data Management (**benchmark/data/**)

- **loaders.py**: Standardized data loading for common BG datasets
- **preprocessors.py**: Data preprocessing and feature engineering utilities
- **validators.py**: Data quality and format validation
- **init.py**: Data module interface

3. Model Framework (**benchmark/models/**)

- **base_model.py**: Abstract base class defining standard model interface
- **deep_learning.py**: Neural network models (LSTM, Transformer, CNN)
- **init.py**: Model module interface

4. Evaluation System (**benchmark/evaluation/**)

- **metrics.py**: Domain-specific evaluation metrics (Clarke EGA, PEGA, TIR)
- **protocols.py**: Standardized evaluation procedures and cross-validation
- **reporting.py**: Result analysis and visualization utilities
- **init.py**: Evaluation module interface

5. Configuration Management (**benchmark/configs/**)

- **config_manager.py**: Configuration loading and validation utilities
- **default.yaml**: Default configuration template
- **example_experiment.yaml**: Complete example configuration

6. Experiment Management (**benchmark/experiments/**)

- **runner.py**: Main experiment execution engine
- **tracking.py**: Experiment logging and reproducibility tracking

7. Results Organization (**benchmark/results/**)

- **README.md**: Results directory structure and format documentation
- Organized structure for experiments, comparisons, and leaderboards

8. Utility Functions (**benchmark/utils/**)

- **logging.py**: Standardized logging configuration
- **io.py**: File I/O and data format utilities
- **visualization.py**: Plotting and visualization functions
- **init.py**: Utilities module interface

9. Data Storage Structure (**data/**)

- **raw/**: Original datasets storage
- **processed/**: Preprocessed datasets
- **metadata/**: Dataset descriptions and schemas
- **README.md**: Data organization and format documentation

10. Project Configuration

- **requirements.txt**: All necessary dependencies for the benchmark
- **setup.py**: Package installation configuration
- **.gitignore**: Appropriate exclusions for ML projects
- **README.md**: Updated main project documentation
- **benchmark/README.md**: Comprehensive benchmark documentation

Key Features Implemented

Reproducibility

- Configuration-driven experiments
- Version control integration
- Standardized interfaces
- Fixed random seeds support

Extensibility

- Modular design for easy extension
- Abstract base classes for consistency
- Plugin-style architecture
- Clear documentation for contributors

Comprehensive Evaluation

- Multiple evaluation metrics
- Cross-validation support
- Statistical significance testing
- Visualization and reporting

Multi-Model Support

- Traditional ML algorithms
- Deep learning architectures
- Physiological models
- Hybrid approaches

Dataset Integration

- Support for common BG datasets
- Standardized data formats
- Quality validation
- Custom dataset support

Usage Examples

Running an Experiment

```
python -m benchmark.cli run --config  
benchmark/configs/example_experiment.yaml
```

Comparing Models

```
python -m benchmark.cli compare --experiments exp1/ exp2/ exp3/
```

Analyzing Results

```
python -m benchmark.cli analyze --experiment-dir  
results/experiments/my_experiment/
```

Next Steps for Implementation

1. **Implement Core Classes:** Fill in the TODO items in each module
2. **Add Model Implementations:** Implement specific models following the base interface
3. **Create Dataset Loaders:** Add support for specific datasets
4. **Implement Metrics:** Add domain-specific evaluation metrics
5. **Add Tests:** Create comprehensive test suite
6. **Documentation:** Expand API documentation and tutorials

This structure provides a solid foundation for reproducible blood glucose forecasting research with clear separation of concerns, standardized interfaces, and comprehensive documentation.